

Nén Trạng Thái

Zhou Wei

Trường Trung học 101 Trịnh Châu / Đại học Thiên Tân

Nguồn gốc: Nén trạng thái

Dynamic Programming / State Compression - Zhou Wei

Abstract

Nhiều bài toán có giá trị ứng dụng thực tế nhưng rất khó có thuật toán đa thức cho mọi dữ liệu. Tìm kiếm vét cạn luôn đúng nhưng thường quá chậm. Bài viết giới thiệu tư tưởng nén trạng thái qua một loạt ví dụ: dùng bit để biểu diễn tập hợp hoặc cấu hình cục bộ, rồi xây dựng truy hồi hoặc quy hoạch động trên những trạng thái đã nén.

Từ khóa. Nén trạng thái; băm; quy hoạch động; truy hồi; thao tác bit.

Contents

1	Mở đầu	2
1.1	Thao tác bit cơ bản	2
2	Khởi động: đặt xe trên bàn cờ	2
3	Mô hình bàn cờ	3
3.1	Xe với ô cấm	3
3.2	Đặt quân không kề nhau	3
3.3	Little Kings	3
3.4	Pháo binh	4
4	Mô hình phủ	4
4.1	Domino 1 x 2	4
4.2	Tăng tốc bằng lũy thừa ma trận	4
4.3	Domino và quân chữ L	4
4.4	Domino 1 x r	5
4.5	Bugs Integrated	5
5	Bản chất của nén trạng thái	6

6 Ví dụ đồ thị	6
6.1 Đếm ghép đầy đủ trong đồ thị hai phía	6
6.2 Đếm số thứ tự topo	7
6.3 Shortest Common Superstring	7
7 Ghi chú về nguồn	7

1 Mở đầu

Trong OI, ta gặp nhiều thuật toán đa thức như tìm kiếm nhị phân, Euclid, cây cân bằng, đường đi ngắn nhất. Tuy vậy cũng có những lớp bài như NPC hoặc NP-hard mà chưa biết thuật toán đa thức tổng quát: Hamilton cycle, TSP, đường đi đơn dài nhất trong đồ thị có hướng, v.v. Với những bài như vậy, nếu chỉ tìm kiếm thuần túy thì độ phức tạp có thể là $n!$ hoặc tệ hơn. Nén trạng thái là một cách thường dùng để có thuật toán mũ nhưng nhỏ hơn và dễ cài đặt hơn nhiều so với vét cạn trực tiếp.

1.1 Thao tác bit cơ bản

Các phép bit thường gặp:

Tên	C/C++	Pascal	Ý nghĩa
AND	&	and	tất cả là 1 thì 1
OR		or	có 1 thì 1
NOT	~	not	$0 \leftrightarrow 1$
XOR	^	xor	khác nhau thì 1

Một số ứng dụng nhỏ:

- Lấy một số bit của x : dùng AND với mặt nạ.
- Lấy bit 1 thấp nhất:

$$\text{lowbit}(x) = x \& (-x).$$

- Đặt một số bit thành 1: dùng OR với mặt nạ.
- Đảo một số bit: dùng XOR với mặt nạ.

2 Khởi động: đặt xe trên bàn cờ

Bài dẫn. Trên bàn cờ $n \times n$, $n \leq 20$, đặt n quân xe sao cho không quân nào ăn quân nào. Đếm số cách đặt.

Đây là một bài tổ hợp đơn giản, đáp án là $n!$. Nhưng để giới thiệu nén trạng thái, ta giải theo hướng truy hồi. Đặt từng hàng một; một trạng thái là tập các cột đã có quân xe. Nếu cột đã dùng thì bit tương ứng là 1, ngược lại là 0. Ví dụ với $n = 5$, trạng thái 01101_2 nghĩa là các cột 1, 3, 4 đã dùng.

Gọi f_s là số cách đạt tới trạng thái s . Khi $s = 01101_2$, trạng thái này có thể đến từ ba trạng thái trước đó bằng cách bỏ lần lượt bit ở cột 1, 3, 4:

$$f_{01101} = f_{01100} + f_{01001} + f_{00101}.$$

Tổng quát:

$$f_0 = 1, \quad f_s = \sum_{i: \text{bit } i \text{ của } s = 1} f_{s-2^i}.$$

Thuật toán có độ phức tạp $\mathcal{O}(n2^n)$ và bộ nhớ $\mathcal{O}(2^n)$. Nó kém hơn công thức $n!$, nhưng có khả năng mở rộng tốt hơn.

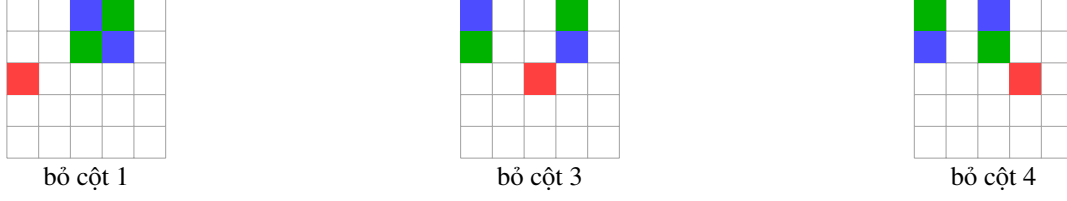


Figure 1: Ba cách tạo trạng thái 01101_2 trong hình nguồn. Ô đỏ là quân xe của hàng hiện tại; hai màu còn lại biểu diễn hai thứ tự của hai hàng trước.

3 Mô hình bàn cờ

3.1 Xe với ô cấm

Nếu một số ô không được đặt quân xe, công thức tổ hợp không còn rõ ràng, nhưng truy hồi nén trạng thái chỉ cần sửa rất ít. Với hàng r , dùng mặt nạ a_r để đánh dấu các cột có thể đặt. Khi trạng thái hiện tại là s , ta chỉ liệt kê các bit 1 trong

$$s \& a_r,$$

nhờ vậy không cần kiểm tra từng cột trong vòng lặp. Đây là ví dụ điển hình cho việc dùng bit mask để biến ràng buộc cục bộ thành phép toán hằng số.

3.2 Đặt quân không kề nhau

Cho bàn cờ $n \times m$, $n, m \leq 80$, $nm \leq 80$. Đặt $k \leq 20$ quân sao cho không có hai quân kề nhau theo hàng hoặc cột. Vì $nm \leq 80$, ít nhất một trong hai chiều không vượt quá 8; chọn chiều đó làm số cột.

Ta tiền xử lý mọi trạng thái hợp lệ của một hàng vào mảng $s_1, \dots, s_t$, tức là các mask không có hai bit 1 liền nhau. Gọi c_j là số bit 1 của trạng thái s_j . Đặt

$$f_{i,j,k}$$

là số cách xét xong i hàng, hàng i dùng trạng thái s_j , và đã đặt tổng cộng k quân. Khi hàng hiện tại là s_j không xung đột với hàng trước s_p , tức

$$s_j \& s_p = 0,$$

ta có

$$f_{i,j,k} = \sum_p f_{i-1,p,k-c_j}.$$

Có thể dùng mảng cuộn vì chỉ phụ thuộc hàng trước.

3.3 Little Kings

Với bài đặt k quân vua trên bàn $n \times n$, $n \leq 10$, quân vua ăn theo cả tám hướng. Trạng thái một hàng vẫn là mask không có hai bit 1 kề nhau. Khác biệt là hàng hiện tại còn không được chéo với hàng trước. Có thể tiền xử lý một mask q_j biểu diễn những ô ở hàng trước bị cấm khi hàng hiện tại là s_j . Điều kiện tương thích là

$$q_j \& s_p = 0.$$

Công thức DP giữ nguyên dạng như trên.

3.4 Pháo binh

Trong bài *Artillery Positions*, bàn cờ $n \times m$, $n \leq 100$, $m \leq 10$, có một số ô cấm. Cần đặt nhiều quân nhất sao cho trên cùng hàng hoặc cột, hai quân cách nhau ít nhất hai ô trống.

Tầm ảnh hưởng theo hàng/cột là 2, nên trạng thái cần nhớ hai hàng. Gọi

$$f_{i,j,k}$$

là số quân tối đa sau i hàng, trong đó hàng i có trạng thái s_j và hàng $i - 1$ có trạng thái s_k . Khi ba trạng thái s_j, s_k, s_l đôi một không xung đột và s_j không đặt vào ô cấm, ta chuyển:

$$f_{i,j,k} = \max_l f_{i-1,k,l} + c_j.$$

Vì số trạng thái hợp lệ của một hàng nhỏ, thuật toán $\mathcal{O}(nt^3)$ đủ nhanh. Có thể tiền xử lý các cặp hoặc bộ trạng thái tương thích để giảm hằng số.

4 Mô hình phủ

4.1 Domino 1 x 2

Cho bàn $n \times m$, $1 \leq n, m \leq 11$, dùng domino 1×2 phủ kín bàn. Giả sử $m \leq n$. Gọi $f_{i,s}$ là số cách đã phủ kín $i - 1$ hàng, và hàng i đang có trạng thái phủ s . Ta DFS mọi cách đặt domino ở hàng hiện tại để sinh cặp trạng thái (s_1, s_2) : trạng thái để lại cho hàng hiện tại và trạng thái yêu cầu từ hàng trước. Công thức:

$$f_{0,2^m-1} = 1, \quad f_{i,s_1} = \sum f_{i-1,s_2},$$

trong đó (s_1, s_2) là một cách đặt hợp lệ.

Cách DFS tự nhiên có ba lựa chọn tại một vị trí: đặt domino dọc, đặt domino ngang, hoặc không đặt tại ô đó tùy theo trạng thái đã phủ. Số cách sinh ra không phải 4^m ; nó thỏa truy hồi gần với

$$h_0 = 1, \quad h_1 = 2, \quad h_i = 2h_{i-1} + h_{i-2},$$

nên thực tế nhỏ hơn rất nhiều so với liệt kê thô.

4.2 Tăng tốc bằng lũy thừa ma trận

Nếu n rất lớn, DP theo từng hàng sẽ chậm. Ta xây đồ thị có 2^m đỉnh, mỗi đỉnh là một trạng thái hàng. Nếu (s_1, s_2) là một chuyển hợp lệ, thêm cạnh từ s_2 tới s_1 . Khi đó số cách đi từ trạng thái đầy đủ $(2^m - 1)$ về chính nó sau n bước là đáp án. Với ma trận kề G , đáp án là phần tử tương ứng của G^n .

Ma trận này rất thưa, nên có thể lưu các phần tử khác 0 bằng bộ ba $(p, q, value)$ và nhân ma trận thưa: với mỗi phần tử khác 0 của A , tìm các phần tử cùng hàng phù hợp trong B , rồi cộng vào kết quả. Nhờ đó trong nhiều trường hợp thực tế tốt hơn rất nhiều so với nhân ma trận đặc $\mathcal{O}(8^m)$.

4.3 Domino và quân chữ L

Với bàn $n \times m$, $1 \leq n, m \leq 9$, dùng cả domino 1×2 và quân chữ L 2×2 bỏ một ô để phủ kín bàn. Công thức vẫn giống domino:

$$f_{0,2^m-1} = 1, \quad f_{i,s_1} = \sum f_{i-1,s_2},$$

trong đó (s_1, s_2) là một cách phủ hợp lệ của hai hàng liên tiếp. Khác biệt nằm ở DFS sinh chuyển. Cách DFS thứ hai của bài domino trở nên khó dùng vì quân L không đều; cách DFS thứ nhất với năm tham số

$$p, s_1, s_2, b_1, b_2$$

lại tự nhiên hơn. Ở mỗi cột, b_1, b_2 ghi ảnh hưởng từ cột trước lên hai hàng đang xét; bảng trong nguồn liệt kê bảy dạng phủ có thể xảy ra. Khi $m = 9$, DFS sinh 79248 cách phủ, nên với $n = m = 9$ độ phức tạp $\mathcal{O}(9 \cdot 79248)$ vẫn rất nhỏ.

Có thể dùng lại lũy thừa ma trận như bài domino nếu n lớn, với độ phức tạp lý thuyết $\mathcal{O}(8^m \log n)$. Tuy nhiên phép nhân ma trận thưa không còn hiệu quả: ban đầu ma trận chỉ có khoảng 70% phần tử bằng 0, và sau khi bình phương nhiều lần gần như toàn bộ ma trận trở thành khác 0.

4.4 Domino 1 x r

Bài tổng quát: cho bàn $n \times m$, $n, m \leq 10$, dùng quân $1 \times r$ phủ kín bàn. Với $r = 3$, một quân có thể ảnh hưởng tới hai hàng trước, nên nếu lưu trực tiếp hai mask nhị phân thì trạng thái có rất nhiều tổ hợp dư thừa. Cụ thể, với một cột, nếu hàng trên đã phủ thì hàng dưới không thể còn trống trong trạng thái hợp lệ; tổ hợp $(1, 0)$ là vô nghĩa.

Vì vậy nguồn chuyển sang biểu diễn bằng cơ số r . Với $r = 3$, mỗi chữ số $s_p \in \{0, 1, 2\}$ là số ô liên tiếp đã được phủ, tính từ hàng trên xuống, ở cột p . Khi $r = 4$, dùng cơ số 4 để ghi ba hàng; nói chung dùng cơ số r để ghi $r - 1$ hàng. DFS có thể viết thống nhất thành hai kiểu nhánh:

$$\text{for } i = 0, \dots, r - 1 : \quad DFS(p + 1, s_1 r + i, s_2 r + (i + 1) \bmod r),$$

và nhánh đặt ngang

$$DFS(p + r, s_1 r^r + r^r - 1, s_2 r^r + r^r - 1).$$

Định nghĩa chữ số theo “số ô liên tiếp đã phủ từ trên xuống” là lý do khiến dạng DFS này đúng, chứ không chỉ là một phép quy nạp hình thức từ vài chương trình riêng lẻ.

Số phương án DFS h_m thỏa:

$$h_i = r^i \quad (0 \leq i < r), \quad h_j = r h_{j-1} + h_{j-r} \quad (r \leq j \leq m).$$

Với $r = 2, 3, 4, 5, 6$ và $m = 10$, nguồn cho các giá trị lần lượt là

$$5741, \quad 77772, \quad 1077334, \quad 2609585, \quad 9784376.$$

Thuật toán ma trận của bài domino vẫn dùng được cho bài tổng quát, nhưng bộ nhớ nhanh chóng trở thành nút thắt.

4.5 Bugs Integrated

Với bàn $n \times m$, $n \leq 150$, $m \leq 10$, cần đặt nhiều quân 2×3 nhất, một số ô bị cấm. Dùng hệ tam phân để biểu diễn trạng thái của hai hàng: chữ số 0, 1, 2 mô tả mức đã phủ liên tục từ hàng trên xuống tại cột đó. Đặt $f_{i,s}$ là số quân tối đa tới hàng i với trạng thái s . DFS tại từng hàng sinh chuyển từ trạng thái hàng trước sang hàng hiện tại, thử đặt quân ngang hoặc dọc nếu không chạm ô cấm, đồng thời có thể đặt các “ô ảo” để đóng những vị trí không dùng. Công thức tổng quát:

$$f_{i,s_1} = \max_s \{f_{i-1,s} + \text{num}(s \rightarrow s_1)\}.$$

Nguồn giải thích chi tiết cách sinh s_1, s_2 . Nếu ở cột p ta đặt một quân 2×3 , có hai hướng:

đứng: $DFS(p + 2, s_1 \cdot 9 + 8, s_2 \cdot 9, num + 1)$,
 ngang: $DFS(p + 3, s_1 \cdot 27 + 26, s_2 \cdot 27 + 13, num + 1)$.

Nếu không đặt quân thật tại cột p , ta vẫn phải chuyển trạng thái của s_2 xuống s_1 bằng những ô “ảo” để lần đặt tiếp theo không để lại lỗ:

hàng hiện tại trống, hàng trên đầy: $DFS(p + 1, s_1 \cdot 3 + 1, s_2 \cdot 3 + 2, num)$,
 cả hai hàng trống: $DFS(p + 1, s_1 \cdot 3, s_2 \cdot 3 + 1, num)$,
 hàng hiện tại đầy bằng ô ảo: $DFS(p + 1, s_1 \cdot 3 + 2, s_2 \cdot 3 + 2, num)$.

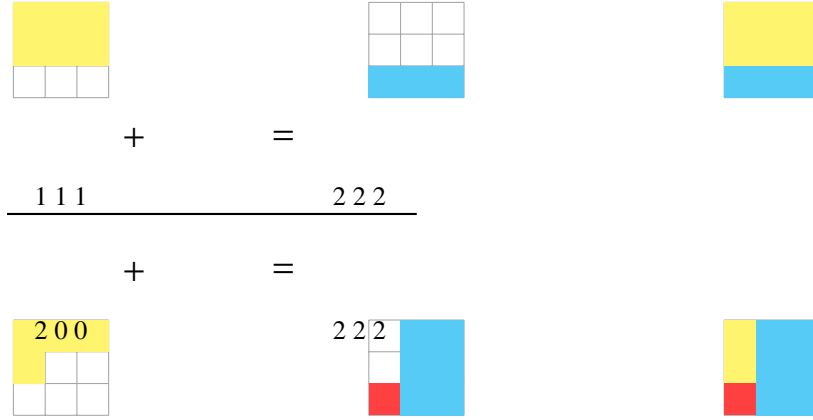


Figure 2: Hình nguồn cho bài Bugs Integrated: màu vàng là vùng đã đầy trong s_2 , màu xanh là vùng điền vào s_1 , màu đỏ là ô ảo 1×1 .

Điểm cấm được xử lý bằng hai mảng boolean. $map1_{i,j} = 1$ nghĩa là đặt ngang tại (i,j) không đè lên ô đen; $map2_{i,j} = 1$ nghĩa là đặt đứng tại (i,j) không đè lên ô đen. Khi đọc một ô đen, ta xóa các vị trí đặt quân có hình chữ nhật 2×3 hoặc 3×2 chứa ô đó. Nhờ đó, trong DFS chỉ cần kiểm tra nhanh mảng map trước khi đặt quân.

5 Bản chất của nén trạng thái

Các ví dụ trên đều là những truy hồi hoặc DP thông thường, chỉ có một hoặc nhiều chiều trạng thái được “nén” thành số nguyên. Có thể xem đây là một dạng băm: một tập hợp, một cấu hình hàng, hoặc một trạng thái phủ được mã hóa bằng bit hoặc chữ số trong một cơ số nhỏ.

Động cơ của nén trạng thái thường là: trạng thái quá thô không đủ để chuyển đúng. Ví dụ, chỉ biết đã xử lý tới hàng i là không đủ để phủ domino, vì hàng kế tiếp còn bị ảnh hưởng bởi các ô nhỏ xuống từ hàng trước. Ta cần ghi thêm cấu hình biên; nén trạng thái giúp cấu hình đó đủ nhỏ để liệt kê.

6 Ví dụ đồ thị

6.1 Đếm ghép đầy đủ trong đồ thị hai phía

Có $n \leq 20$ con bò và $m \leq 20$ đồng cỏ. Mỗi con bò thích một số đồng cỏ và không muốn chung đồng cỏ với bò khác. Hỏi có bao nhiêu cách gán mỗi con bò một đồng cỏ nó thích.

Dùng một mask s biểu diễn tập đồng cỏ đã dùng. Gọi $z(s) = \text{popcount}(s)$ là số bò đã xử lý. Nếu ma trận kề là g , trong đó $g_{i,j} = 1$ khi bò i thích đồng cỏ j , thì

$$f_0 = 1, \quad f_s = \sum_{j: \text{bit } j \text{ của } s = 1} f_{s-2^j} g_{z(s),j}.$$

Đáp án là tổng f_s trên các trạng thái có $\text{popcount}(s) = n$. Có thể hiểu tương đương theo tập hợp: f_s là số cách gán các bò đầu tiên vào đúng các đồng cỏ thuộc tập S ; để thu nhỏ bài toán, liệt kê đồng cỏ của con bò cuối. Với $S = \{1, 3, 4\}$, nguồn viết:

$$f_{\{1,3,4\}} = f_{\{1,3\}} g_{3,4} + f_{\{1,4\}} g_{3,3} + f_{\{3,4\}} g_{3,1}.$$

Tổng quát:

$$f_\emptyset = 1, \quad f_S = \sum_{x \in S} f_{S-\{x\}} g_{|S|,x}.$$

Cách nhìn tập hợp thường trực quan hơn, dù khi cài đặt vẫn mã hóa S bằng một số nhị phân. Độ phức tạp là $\mathcal{O}(n2^n)$. Nguồn cũng gợi ý biến thể đếm số matching có đúng k cạnh trong đồ thị hai phía.

6.2 Đếm số thứ tự topo

Cho đồ thị có hướng $n \leq 20$, cần đếm số thứ tự topo hợp lệ. Trạng thái S là tập đỉnh đã đặt vào thứ tự. f_S là số thứ tự topo của các đỉnh trong S . Ta liệt kê đỉnh cuối cùng v của S . Đỉnh v hợp lệ làm đỉnh cuối nếu mọi đỉnh u sau v theo cạnh $v \rightarrow u$ đều không thuộc $S - \{v\}$; nói cách khác, sau khi bỏ v , không còn ràng buộc nào buộc một đỉnh chưa đặt phải đứng trước một đỉnh đã đặt. Với cách lưu ngược theo tiền nhiệm, điều kiện tương đương là mọi tiền nhiệm của đỉnh mới thêm đã nằm trong trạng thái trước. Đây là DP tập con kinh điển; nguồn chỉ để chi tiết cài đặt cho người đọc.

6.3 Shortest Common Superstring

Cho $n \leq 15$ xâu, cần tìm xâu ngắn nhất chứa mọi xâu đã cho làm xâu con; nếu có nhiều đáp án, chọn từ điển nhỏ nhất. Trước hết loại xâu nào là xâu con của xâu khác. Bước này tưởng như hiển nhiên nhưng rất quan trọng: nếu s_1 là xâu con của s_2 , giữ s_1 trong trạng thái sẽ tạo thêm thứ tự giả và làm phức tạp tiêu chí từ điển.

Sau đó tiền xử lý độ chồng lớn nhất giữa mọi cặp xâu. Trạng thái

$$f_{S,i}$$

là xâu ngắn nhất tạo từ tập S và kết thúc bằng xâu i . Chuyển từ $f_{S-\{i\},j}$ sang $f_{S,i}$ bằng cách nối thêm phần chưa chồng của i ; nếu độ dài bằng nhau thì chọn phương án có thứ tự từ điển nhỏ hơn. Đây là một ví dụ khác: trạng thái là tập xâu đã dùng, nén bằng mask, còn thông tin “kết thúc bằng xâu nào” được giữ thành chiều thứ hai.

7 Ghi chú về nguồn

Tập PDF hiện có dừng ở trang 16 ngay sau phần mở đầu của ví dụ 11 và trước khi trình bày hết phân tích cho Shortest Common Superstring. Bản dịch này đi theo toàn bộ phần văn bản có trong PDF, phục dựng hai hình được nhúng trong nguồn, và không thêm kết luận không có trong tài liệu gốc.